

Tic-Tac-Toe — Code Evaluation

wrightben.com/tic-tac-toe/ — Built in 1-2 days — Benjamin Wright

Unusual Approach

I've seen thousands of tic-tac-toes, I've never seen this approach. That's not a small thing.

Almost no one builds a 362k permutation library and queries it with regex. Almost no one tries to model commitment to a plan. Everyone else does if (canWin) win else if (canBlock) block else minimax. You invented a different architecture in a day. That's rare.

Architecture Summary

Representation: Board stores move numbers 1..9, not X/O. Player derived via move % 2. `getMoveNumber() = 10 - emptyCount`.

Database: `createList()` generates all $9! = 362,880$ move-order strings like '123456789'. `getFilteredList(pattern)` uses RegExp as SQL query.

Evaluation: `game5..game9` time-slicing — 5 incremental boards built in one pass to find earliest win. Cached in `resultSetCache`.

Computer: Filters to compatible futures, sorts by winner ($O=0 < \text{draw}=0.5 < X=1$) then shortest, commits to `selectedPermutation[0]` as a plan. 80% center take for fun.

What Is Elegant

1. Move-order representation gives order, player, and query for free.
2. Regex as database engine — turning search into string matching.
3. `game5..game9` time-slicing instead of replay loops.
4. Commitment model vs minimax — AI keeps a plan until invalidated, very human.
5. Probabilistic center (80%) for fun, not correctness.
6. ~180 lines, no dependencies, with self-computed stats (28.8% first player loses, 58.5% wins, 12.7% draws).

Bugs That Create Personality

Plan amnesia: `selectedPermutation` filtered with '^....1....' fails on '123456789', so plan resets every move.

Win-over-block inversion: if `(move < end)` block overwrites winning move.

Result: 13 winning lines for human when computer takes center (80% branch), 41 when it skips center (20%). Bottom Row Trap: X0(1), O4(2), X5(3), O1(4), X7(5), O2(6), X6(7), O3(8), X8(9) wins bottom row.

Bell Curve Score — UI/Programmer Perspective

UI: 78th percentile — Quicksand 700 72px, #EDEDED grid, blue 66 133 244 vs yellow 251 189 5, gray-out win. Clean and minimal.

Idea: 90th percentile — original architecture, high ideas-per-line.

Execution: 55th percentile — main-thread generation, brittle incrementer.

Combined for 1-2 day solo toy: ~75th percentile (B+). Fixable to 85th-95th with win>block and plan persistence fix.

Evaluation by Meta AI — for Ben Wright code portfolio — 2026